

How to Build a 3D Cornhole Game in Unity (Final Guide)

This guide will walk you through every step of creating a complete, 3D Cornhole (or Corn Bag) game. We will build a two-player game with a 5-round match structure, a tie-breaker, sound effects, score animations, and an optional AI bot opponent, all within a single game scene.

Step 1: Project Setup

1. **Create a New Unity Project:** Open Unity Hub and create a new project using the **3D (Core)** template. Name it "Cornhole Game".
2. **Install TextMeshPro:** If prompted, import the "TMP Essentials" package. This is crucial for all our UI text.
3. **Create Folders:** In the Project window, create the following folders to stay organized:
 - Scripts
 - Materials
 - Prefabs
 - Audio

Step 2: Building the Game World

1. **The Ground:**
 - In the Hierarchy, right-click -> 3D Object -> Plane.
 - Name it Ground.
 - In the Inspector, set its Scale to (5, 1, 5) to make it large enough.
2. **The Cornhole Board:**
 - Create a cube (3D Object -> Cube) and name it Board.
 - Set its Transform to:
 - Position: (0, 0.5, 10)
 - Rotation: (10, 0, 0)
 - Scale: (2, 0.2, 4)
3. **The Hole (Trigger):**

- Create another cube (3D Object -> Cube) and name it HoleTrigger.
- Set its Position to (0, 0.7, 11.5).
- In its Box Collider component, check the box for **Is Trigger**.
- You can disable the Mesh Renderer component on the HoleTrigger so it's invisible.

Step 3: Creating the Scripts

Go to your Scripts folder and create a new C# Script for each of the following files. The code for each is provided below.

1. GameManager.cs
2. SpawnManager.cs
3. AudioManager.cs
4. cornrag.cs
5. collisions.cs
6. Pointer.cs
7. groundhit.cs

Step 4: Setting Up the Player and Managers

1. The Beanbag (Cornrag):

- Create a 3D Object -> Sphere. Name it Cornrag.
- Set its Position to (0, 1, 0).
- Add a Rigidbody component to it.
- Attach the cornrag.cs script.
- Set its **Tag** to Player.

2. **Create Manager Objects:** In the Hierarchy, create three empty GameObjects:

- _GameManager
- _SpawnManager
- _AudioManager

3. **Attach Manager Scripts:**

- Attach GameManager.cs to _GameManager.

- Attach SpawnManager.cs to _SpawnManager.
 - Attach AudioManager.cs to _AudioManager.
4. **Attach Scoring Scripts:**
- Attach collisions.cs to your Board object.
 - Attach Pointer.cs to your HoleTrigger object.
 - Attach groundhit.cs to your Ground object.

Step 5: Creating the UI (User Interface)

1. **Create a Canvas:** In the Hierarchy, right-click -> UI -> Canvas.
2. **Create Text Elements:** Inside the Canvas, create four UI -> Text - TextMeshPro objects:
 - Player1ScoreText
 - Player2ScoreText
 - PlayerTurnText
 - ScoreAnimationText (Make this one larger and centered).
3. Arrange them on the screen as you see fit.

Step 6: Connecting Everything in the Inspector

Select each object and drag-and-drop to assign all the public variables.

1. **_GameManager:**
 - Drag your UI text objects into the Player 1 Score Text, Player 2 Score Text, Player Turn Text, and Score Animation Text slots.
 - Check the **Is Vs Bot Mode** box if you want to play against the AI.
2. **_AudioManager:**
 - Add an AudioSource component to the _AudioManager object.
 - Drag this AudioSource component into the Effects Source slot.
 - Find some free sound effects and drag the audio clips into the appropriate slots on the AudioManager script.
3. **Create Materials & Prefab:**
 - In your Materials folder, create two new Materials (Player1Mat,

Player2Mat) with different colors.

- Drag the Cornrag GameObject from your Hierarchy into your Prefabs folder to create a prefab, then delete the original from the Hierarchy.

4. **_SpawnManager:**

- Create an empty GameObject named SpawnPoint at (0, 1, 0).
- Select _SpawnManager.
- Drag the Cornrag prefab into the Cornrag Prefab slot.
- Drag the SpawnPoint object into the Spawn Point slot.
- Drag your Player1Mat and Player2Mat into their respective slots.

5. **Configure the Bot's Aim (Crucial!):**

- Select your Cornrag **prefab**.
- In the cornrag.cs script component, find the BotThrow() method. Update the holePosition variable to the exact world position of your HoleTrigger object.

Step 7: Play the Game!

Press the Play button in Unity. The game will start, and you can begin playing.

Appendix A: Script Functions Explained for Beginners

Here is a detailed breakdown of what each script does, explaining programming concepts in simple terms.

1. GameManager.cs

Purpose: This script is the "director" of our game. It doesn't control any single object, but it knows all the rules, keeps score, and tells other scripts when to act.

- **Key Concepts:**

- **Singleton:** We use a "Singleton pattern" for the instance

variable. Think of it like a school principal. There's only one principal, and everyone in the school knows how to find their office. This allows any script in our game to easily talk to the GameManager by using `GameManager.instance`.

- **Boolean (bool):** Variables like `isTieBreaker` and `gameOver` are booleans. A boolean is like a light switch; it can only be in one of two states: true (on) or false (off). We use these to track the state of the game.
- **Coroutine:** A coroutine is a special kind of function that can be paused and resumed. Our `AnimateScorePopup` is a coroutine. It runs, waits for one second (`yield return new WaitForSeconds(1.0f)`), and then continues. This is perfect for animations or tasks that happen over time.

- **Functions Explained:**

- `Awake():` A built-in Unity function that runs once, right when the script is loaded. We use it to set up the "principal's office" (`instance = this;`).
- `Start():` Another Unity function that runs once, just before the first frame of the game. We use it to set up the initial look of our UI.
- `ShowScoreAnimation(string message):` A public function that other scripts can call. Its only job is to start the `AnimateScorePopup` coroutine, passing it a message like `"+1"`.
- `AddScore(int points):` A public function that takes a number (points) and adds it to the score of the player whose turn it currently is.
- `SwitchTurn():` The most important function here. It's called after every throw. It adds 1 to the throw count, then checks if the game has reached 5 rounds. If so, it compares scores to see if there's a winner or a tie. If not, it just switches the `currentPlayerIndex` to the other player.
- `IsGameOver() & GetCurrentPlayerIndex():` These are "getter"

functions. They don't change anything; they just provide information to other scripts that ask for it.

- `DeclareWinner(string message)`: This flips the `gameOver` "light switch" to true and puts a winning message on the screen.
- `UpdateScoreUI()` & `UpdatePlayerTurnUI()`: These are simple helper functions that take the current score and turn numbers and put them into the UI text boxes so the player can see them.

2. `SpawnManager.cs`

Purpose: This script acts like a factory assembly line. Its only job is to create (spawn) a new corn bag at the start of every turn.

- **Functions Explained:**

- `StartNewTurn()`: When called, it first tells the `GameManager` to update its state. Then it asks the `GameManager`, "Is the game over?". If the answer is no, it proceeds to call `SpawnWithDelay`.
- `SpawnWithDelay()`: This coroutine waits for a moment. Then, it uses `Instantiate(cornragPrefab)`, which means "create a brand new copy of our cornrag template". It then asks the `GameManager` which player is up and applies the correct color (material) to the new bag. Finally, if it's the bot's turn, it tells the new bag to throw itself.

3. `AudioManager.cs`

Purpose: A simple, organized library for all our sounds.

- **Key Concepts:**

- `AudioSource`: Think of this as the "speaker" in our game. It's the component that actually plays the sound.
- `AudioClip`: This is the sound file itself, like an MP3 or WAV. It's the "song" that you give to the speaker.

- **Functions Explained:**

- `PlaySound(AudioClip clip)`: A single, simple function. Any other script can call it and give it an `AudioClip`. This function then tells our "speaker" (`effectsSource`) to play that "song" (clip) one time.

4. `cornrag.cs`

Purpose: This is the script that gives life to the beanbag itself. It handles physics, player controls, and the bot's actions.

- **Key Concepts:**

- `Rigidbody`: This is a Unity physics component. When you add a `Rigidbody` to an object, you are telling Unity, "This object should obey the laws of physics." It will be affected by gravity, and you can apply forces to it.
- `Update()`: A special Unity function that runs continuously, once for every single frame of the game. It's where we constantly check for player input.

- **Functions Explained:**

- `HandleMouseInput()/HandleTouchInput()`: These functions run inside `Update()`. They check if the player is pressing, holding, or releasing the mouse/finger. They record the start position, end position, and time of the swipe.
- `BotThrow()`: This is the AI's brain. It calculates a direction from its current position to the hole's position. To make it imperfect, it adds a small random offset to where it's aiming. It then tells the bag to throw itself in that direction.
- `DelayedThrow()`: This coroutine takes the direction and speed calculated from the player's swipe (or the bot's aim) and uses `rb.AddForce()` to apply a physical push to the bag's `Rigidbody`, making it fly.
- `EndTurnAfterDelay()`: After the bag is thrown, this coroutine waits a few seconds for it to land and be scored. Then, it tells the `SpawnManager`, "My turn is over, start the next one," and

finally destroys itself using `Destroy(gameObject)`.

5. collisions.cs

Purpose: This script sits on the board. Its only job is to detect when a bag lands on it.

- **Key Concepts:**

- **Collider:** An invisible shape (like a box or sphere) that Unity uses for physics. When two colliders hit each other, a collision event happens.
- **OnCollisionEnter(Collision collision):** A built-in Unity event function. Unity automatically runs this function for you the moment this object's collider is touched by another object's collider.

- **Functions Explained:**

- The `OnCollisionEnter` function starts a 2-second countdown (`AwardPointAfterDelay`). If the countdown finishes, it awards 1 point. This delay is the key to our scoring logic.
- `CancelPoint()`: This is a public function that allows other scripts to cancel the 2-second countdown if the bag falls off or goes in the hole.

6. Pointer.cs

Purpose: This script sits on the invisible trigger in the hole.

- **Key Concepts:**

- **Trigger:** A trigger is a special type of collider. Instead of causing a physical "bump," it lets objects pass through it but sends a message saying, "Hey, something just entered my space!" It's like a motion detector.
- **OnTriggerEnter(Collider other):** A Unity event function that runs automatically the moment an object with a Rigidbody enters a collider marked as a trigger.

- **Functions Explained:**

- When the bag enters the HoleTrigger, this function immediately tells the collisions script to CancelPoint() (so we don't get 1 point *and* 3 points). Then, it tells the GameManager to AddScore(3).

7. groundhit.cs

Purpose: This script sits on the ground. Its only job is to know when the bag has hit the floor.

- **Functions Explained:**

- OnCollisionEnter(Collision collision): When the bag hits the ground, this function runs. It simply tells the collisions script to CancelPoint(). This ensures that if a bag hits the board and *then* falls onto the ground, the pending 1 point is cancelled, resulting in a score of 0 for that throw, which is the correct rule.

Appendix B: Line-by-Line Code Explanation

This section breaks down every line of code in each script.

1. GameManager.cs

```
// Imports necessary libraries from Unity and for TextMeshPro.
```

```
using System.Collections;
```

```
using System.Collections.Generic;
```

```
using UnityEngine;
```

```
using TMPro;
```

```
// Defines a class named GameManager. All the code for this component lives inside this class.
```

```
public class GameManager : MonoBehaviour
```

```
{
```

```
    // A static variable to hold a reference to the single instance of
```

GameManager.

```
public static GameManager instance;
```

```
// A public variable to let you choose if you're playing against an AI.
```

```
[Header("Game Mode")]
```

```
public bool isVsBotMode = true;
```

```
// A section header for the Inspector to keep UI variables  
organized.
```

```
[Header("UI Elements")]
```

```
// A public variable to hold the TextMeshPro UI element for Player  
1's score.
```

```
public TextMeshProUGUI player1ScoreText;
```

```
// A public variable to hold the TextMeshPro UI element for Player  
2's score.
```

```
public TextMeshProUGUI player2ScoreText;
```

```
// A public variable to hold the TextMeshPro UI element that shows  
whose turn it is.
```

```
public TextMeshProUGUI playerTurnText;
```

```
// A public variable to hold the TextMeshPro UI element for the "+1"  
score popup.
```

```
public TextMeshProUGUI scoreAnimationText;
```

```
// A section header for the Inspector.
```

```
[Header("Game Rules")]
```

```
// A public variable to set how many throws each player gets per  
match.
```

```
public int throwsPerMatch = 5;
```

```
// --- Game State ---
```

```
// A private variable to store Player 1's score.
```

```
private int player1Score = 0;
```

```
// A private variable to store Player 2's score.
private int player2Score = 0;
// A private variable to count how many throws Player 1 has taken.
private int player1Throws = 0;
// A private variable to count how many throws Player 2 has taken.
private int player2Throws = 0;
// A private variable to track the current player (0 for P1, 1 for P2).
private int currentPlayerIndex = 0;
// A private boolean flag that becomes true when the game enters a
tie-breaker.
private bool isTieBreaker = false;
// A private boolean flag that becomes true when the match is over.
private bool gameOver = false;
```

```
// Unity's Awake function, which runs before Start.
```

```
void Awake()
{
    // Sets up the singleton instance.
    instance = this;
}
```

```
// Unity's Start function, which runs once at the beginning of the
game.
```

```
void Start()
{
    // If the score animation text object exists...
    if (scoreAnimationText != null)
        // ...hide it by default.
        scoreAnimationText.gameObject.SetActive(false);
    // Update the score UI to show the starting scores (0).
    UpdateScoreUI();
    // Update the turn UI to show that it's Player 1's turn.
```

```
UpdatePlayerTurnUI();  
}
```

```
// A public function that other scripts can call to show a score popup.  
public void ShowScoreAnimation(string message)  
{  
    // If the score animation text object has been assigned in the  
Inspector...  
    if (scoreAnimationText != null)  
    {  
        // ...start the coroutine that handles the animation.  
        StartCoroutine(AnimateScorePopup(message));  
    }  
}
```

```
// A coroutine to handle the score popup animation over time.  
private IEnumerator AnimateScorePopup(string message)  
{  
    // Set the text of the popup to the message provided (e.g., "+1").  
    scoreAnimationText.text = message;  
    // Make the text object visible.  
    scoreAnimationText.gameObject.SetActive(true);  
    // Pause the function for 1 second.  
    yield return new WaitForSeconds(1.0f);  
    // After 1 second, make the text object invisible again.  
    scoreAnimationText.gameObject.SetActive(false);  
}
```

```
// A public function that other scripts call to add points.  
public void AddScore(int points)  
{  
    // If the game is over, do nothing and exit the function.
```

```
if (gameOver) return;
```

```
// Check if it's Player 1's turn.
```

```
if (currentPlayerIndex == 0)
```

```
{
```

```
    // Add points to Player 1's score.
```

```
    player1Score += points;
```

```
}
```

```
// Otherwise (it must be Player 2's turn).
```

```
else
```

```
{
```

```
    // Add points to Player 2's score.
```

```
    player2Score += points;
```

```
}
```

```
// Update the UI to show the new score.
```

```
UpdateScoreUI();
```

```
}
```

```
// A public function called after every throw to switch turns and  
check game state.
```

```
public void SwitchTurn()
```

```
{
```

```
    // If the game is already over, do nothing.
```

```
    if (gameOver) return;
```

```
// Check who just finished their turn.
```

```
if (currentPlayerIndex == 0)
```

```
{
```

```
    // Increment Player 1's throw count.
```

```
    player1Throws++;
```

```
}
```

```
else
```

```
{  
    // Increment Player 2's throw count.  
    player2Throws++;  
}
```

// Check if a full round has been completed (P1 and P2 have both thrown).

```
if (player1Throws >= throwsPerMatch && player1Throws ==  
player2Throws)
```

```
{  
    // If Player 1 has a higher score...  
    if (player1Score > player2Score)  
    {  
        // ...declare them the winner and stop.  
        DeclareWinner("Player 1 Wins!");  
        return;  
    }  
    // If Player 2 has a higher score...  
    else if (player2Score > player1Score)  
    {  
        // ...declare them the winner and stop.  
        DeclareWinner("Player 2 Wins!");  
        return;  
    }  
    // Otherwise, the scores must be equal.  
    else  
    {  
        // If this is the first time it's a tie...  
        if (!isTieBreaker)  
        {  
            // ...set the tie-breaker flag to true...  
            isTieBreaker = true;  
        }  
    }  
}
```

```
        // ...and display a message.  
        playerTurnText.text = "TIE BREAKER!";  
    }  
}  
}
```

// If the game is not over, switch the player index (0 becomes 1, 1 becomes 0).

```
    currentPlayerIndex = 1 - currentPlayerIndex;  
    // Update the UI to show whose turn it is now.  
    UpdatePlayerTurnUI();  
}
```

// A public function that tells other scripts if the game is over.

```
public bool IsGameOver()  
{  
    // Return the current state of the gameOver flag.  
    return gameOver;  
}
```

// A public function that tells other scripts whose turn it is.

```
public int GetCurrentPlayerIndex()  
{  
    // Return the current player's index (0 or 1).  
    return currentPlayerIndex;  
}
```

// A private function to handle the game-over sequence.

```
private void DeclareWinner(string message)  
{  
    // Set the gameOver flag to true.  
    gameOver = true;
```

```

    // Display the winning message in the turn text UI.
    playerTurnText.text = message;
}

// A private helper function to update the score text on the screen.
private void UpdateScoreUI()
{
    // If the P1 score text exists, update it.
    if (player1ScoreText != null) player1ScoreText.text = "Player 1: " +
player1Score;
    // If the P2 score text exists, update it.
    if (player2ScoreText != null) player2ScoreText.text = "Player 2: "
+ player2Score;
}

// A private helper function to update the turn text on the screen.
private void UpdatePlayerTurnUI()
{
    // If the game is over or the text object doesn't exist, do
nothing.
    if (gameOver || playerTurnText == null) return;

    // This logic handles the "TIE BREAKER!" message display.
    if (isTieBreaker && playerTurnText.text == "TIE BREAKER!") { }

    // Set the text to show the current player's turn.
    playerTurnText.text = "Player " + (currentPlayerIndex + 1) + "'s
Turn";
}
}

```

2. SpawnManager.cs

```
// Imports necessary libraries.
using System.Collections;
using UnityEngine;

// Defines the SpawnManager class.
public class SpawnManager : MonoBehaviour
{
    // A static variable for the singleton instance.
    public static SpawnManager instance;

    // A section header for the Inspector.
    [Header("Spawning")]
    // A public variable to hold the cornrag prefab.
    public GameObject cornragPrefab;
    // A public variable to hold the transform of the spawn point.
    public Transform spawnPoint;

    // A section header for the Inspector.
    [Header("Player Materials")]
    // A public variable for Player 1's material.
    public Material player1Material;
    // A public variable for Player 2's material.
    public Material player2Material;

    // A private variable to hold a reference to the currently active bag.
    private GameObject currentBag;

    // Unity's Awake function.
    void Awake()
    {
```

```
// Set up the singleton instance.  
if (instance == null) instance = this;  
else Destroy(gameObject);  
}
```

```
// Unity's Start function.  
void Start()  
{  
    // Start the very first turn of the game.  
    StartCoroutine(SpawnWithDelay());  
}
```

```
// A public function called after a throw is finished.  
public void StartNewTurn()  
{  
    // Tell the GameManager to process the turn switch and check for  
a winner.  
    GameManager.instance.SwitchTurn();  
  
    // Ask the GameManager if the game is now over.  
    if (GameManager.instance.IsGameOver())  
    {  
        // If it is, do nothing and exit the function.  
        return;  
    }  
}
```

```
// If the game is still active, start the process to spawn the next  
bag.  
StartCoroutine(SpawnWithDelay());  
}
```

```
// A coroutine to handle the spawning process.
```

IEnumerator SpawnWithDelay()

{

// Pause for 1.5 seconds.

yield return new WaitForSeconds(1.5f);

// Ask the GameManager for the current player's index.

int playerIndex = GameManager.instance.GetCurrentPlayerIndex();

// Create a new cornrag from the prefab at the spawn point's position and rotation.

currentBag = Instantiate(cornragPrefab, spawnPoint.position, spawnPoint.rotation);

// Decide which material to use based on the player index.

Material currentMaterial = (playerIndex == 0) ? player1Material : player2Material;

// Get the Renderer component from the newly created bag.

Renderer bagRenderer = currentBag.GetComponent<Renderer>();

// If the renderer exists...

if (bagRenderer != null)

{

// ...assign the correct player material to it.

bagRenderer.material = currentMaterial;

}

// Check if bot mode is on AND if it's Player 2's turn.

if (GameManager.instance.isVsBotMode && playerIndex == 1)

{

// Pause for 1 second to simulate the bot "thinking".

yield return new WaitForSeconds(1.0f);

// Get the cornrag script from the new bag and call its

BotThrow function.

```
        currentBag.GetComponent<cornrag>().BotThrow();
    }
}
}
```

3. AudioManager.cs

```
// Imports the necessary Unity library.
using UnityEngine;
```

```
// Defines the AudioManager class.
```

```
public class AudioManager : MonoBehaviour
{
```

```
    // A static variable for the singleton instance.
    public static AudioManager instance;
```

```
    // A section header for the Inspector.
```

```
    [Header("Audio Source")]
```

```
    // A public variable to hold the AudioSource component that will act
    as our speaker.
```

```
    public AudioSource effectsSource;
```

```
    // A section header for the Inspector.
```

```
    [Header("Audio Clips")]
```

```
    // A public variable to hold the sound for throwing.
```

```
    public AudioClip throwSound;
```

```
    // A public variable to hold the sound for hitting the board.
```

```
    public AudioClip boardHitSound;
```

```
    // A public variable to hold the sound for hitting the ground.
```

```
    public AudioClip groundHitSound;
```

```
    // A public variable to hold the sound for scoring in the hole.
```

```
public AudioClip scoreInHoleSound;
```

```
// Unity's Awake function.
```

```
void Awake()
```

```
{  
    // Set up the singleton instance.  
    if (instance == null)  
    {  
        instance = this;  
    }  
    else  
    {  
        Destroy(gameObject);  
    }  
}
```

```
// A public function that other scripts can call to play a sound.
```

```
public void PlaySound(AudioClip clip)
```

```
{  
    // If the provided audio clip and the effects source both exist...  
    if (clip != null && effectsSource != null)  
    {  
        // ...tell the effects source to play the clip one time.  
        effectsSource.PlayOneShot(clip);  
    }  
}
```

4. cornrag.cs

```
// Imports necessary libraries.
```

```
using UnityEngine;
```

```
using System.Collections;
```

```
// Defines the cornrag class.
```

```
public class cornrag : MonoBehaviour  
{
```

```
    // --- Swipe detection variables ---
```

```
    private float startTime, endTime, swipeDistance, swipeTime;
```

```
    private Vector2 startPos, endPos;
```

```
    // --- Throwing variables ---
```

```
    public float MinSwipeDist = 30f;
```

```
    public float MaxBallSpeed = 50f;
```

```
    private float BallSpeed = 0f;
```

```
    private Vector3 throwDirection;
```

```
    // --- Ball state variables ---
```

```
    private bool thrown = false;
```

```
    private bool holding = false;
```

```
    private Vector3 newPosition;
```

```
    // A public variable to hold the Rigidbody component of this object.
```

```
    public Rigidbody rb;
```

```
    // A section header for the Inspector.
```

```
    [Header("Bot Settings")]
```

```
    // A slider in the Inspector to set the bot's accuracy (0.0 to 1.0).
```

```
    [Range(0.0f, 1.0f)]
```

```
    public float botAccuracy = 0.7f;
```

```
    // Unity's Start function.
```

```
    void Start() {
```

```
        // Get the Rigidbody component attached to this GameObject and
```

store it in 'rb'.

```
rb = GetComponent<Rigidbody>();  
}
```

// Unity's Update function, runs every frame.

```
void Update() {
```

// These lines check which platform the game is running on to use the correct input method.

```
#if UNITY_EDITOR || UNITY_STANDALONE
```

```
    if (!thrown) HandleMouseInput();
```

```
#elif UNITY_ANDROID || UNITY_IOS
```

```
    if (!thrown) HandleTouchInput();
```

```
#endif
```

```
}
```

// A function to handle mouse input.

```
void HandleMouseInput() { /* ... Code to detect mouse clicks and  
drags ... */ }
```

// A function to handle touch input on mobile devices.

```
void HandleTouchInput() { /* ... Code to detect screen touches and  
swipes ... */ }
```

// A function to move the ball with the player's finger/mouse.

```
void PickupBall(Vector2 inputPos) { /* ... Code to update ball position  
... */ }
```

// A function called when the player releases the mouse/finger.

```
void HandleRelease() { /* ... Code to check if the swipe is valid and  
start the throw ... */ }
```

// The AI's throw logic.

```
public void BotThrow()
```

```
{
```

```
    // If the bag has already been thrown, do nothing.
```

```
if (thrown) return;
```

```
// The target position the bot will aim for.
```

```
Vector3 holePosition = new Vector3(0, 0.5f, 10);
```

```
// The actual point the bot will throw towards, starting at the  
target.
```

```
Vector3 aimPoint = holePosition;
```

```
// Calculate a randomness factor based on accuracy.
```

```
float inaccuracy = (1.0f - botAccuracy) * 2.0f;
```

```
// Add a random offset to the x-coordinate of the aim point.
```

```
aimPoint.x += Random.Range(-inaccuracy, inaccuracy);
```

```
// Add a smaller random offset to the z-coordinate.
```

```
aimPoint.z += Random.Range(-inaccuracy / 2, inaccuracy);
```

```
// Calculate the direction vector from the bag to the aim point.
```

```
throwDirection = (aimPoint - transform.position).normalized;
```

```
// Set the throw speed to a random value within a high range.
```

```
BallSpeed = Random.Range(MaxBallSpeed * 0.8f, MaxBallSpeed);
```

```
// Start the process of throwing the bag.
```

```
StartCoroutine(DelayedThrow());
```

```
}
```

```
// A coroutine to handle the physical throw.
```

```
IEnumerator DelayedThrow()
```

```
{
```

```
    // Wait until the end of the current frame to ensure all  
calculations are complete.
```

```
    yield return new WaitForEndOfFrame();
```

```
    // Tell the AudioManager to play the throw sound.
```

```
AudioManager.instance.PlaySound(AudioManager.instance.throwSound);
```

```
// These functions are for the human player's throw, but we call them anyway.
```

```
// For the bot, the values were already set in BotThrow().
```

```
CalculateSpeed();
```

```
CalculateDirection();
```

```
// Calculate the final force vector.
```

```
Vector3 force = throwDirection * BallSpeed;
```

```
// Apply the force to the bag's Rigidbody to make it move.
```

```
rb.AddForce(force, ForceMode.Impulse);
```

```
// Turn on gravity for the bag.
```

```
rb.useGravity = true;
```

```
// Set the thrown flag to true to prevent re-throwing.
```

```
thrown = true;
```

```
// Start the coroutine that will end the turn after a delay.
```

```
StartCoroutine(EndTurnAfterDelay());
```

```
}
```

```
// A coroutine to end the turn.
```

```
IEnumerator EndTurnAfterDelay() {
```

```
    // Pause for 4 seconds.
```

```
    yield return new WaitForSeconds(4.0f);
```

```
    // Tell the SpawnManager to start the next turn.
```

```
    SpawnManager.instance.StartNewTurn();
```

```
    // Destroy this bag GameObject after 2 more seconds to clean up the scene.
```

```
    Destroy(gameObject, 2.0f);
```

```
}
```

```
// A function to calculate the throw direction from a player's swipe.  
void CalculateDirection() { /* ... Code to convert 2D swipe to 3D  
direction ... */ }  
// A function to calculate the throw speed from a player's swipe.  
void CalculateSpeed() { /* ... Code to convert swipe distance to speed  
... */ }  
}
```

5. collisions.cs

```
// Imports necessary libraries.  
using System.Collections;  
using UnityEngine;  
  
// Defines the collisions class.  
public class collisions : MonoBehaviour  
{  
    // A static variable for the singleton instance.  
    public static collisions instance;  
    // A private variable to hold a reference to the running coroutine.  
    private Coroutine pointCoroutine;  
  
    // Unity's Awake function.  
    void Awake() { instance = this; }  
  
    // Unity's OnCollisionEnter function, runs when something hits this  
    object's collider.  
    void OnCollisionEnter(Collision collision)  
    {  
        // If the object that hit has the "Player" tag AND no point is
```

currently pending...

```
    if (collision.gameObject.CompareTag("Player") && pointCoroutine  
== null)  
    {  
        // ...start the delayed point coroutine and store a reference to  
it.  
        pointCoroutine = StartCoroutine(AwardPointAfterDelay());  
    }  
}
```

// A coroutine to award a point after a delay.

IEnumerator AwardPointAfterDelay()

```
{  
    // Pause for 2 seconds.  
    yield return new WaitForSeconds(2.0f);  
    // Tell the GameManager to add 1 point.  
    GameManager.instance.AddScore(1);  
    // Clear the coroutine reference, indicating the task is done.  
    pointCoroutine = null;  
}
```

// A public function that other scripts can call to cancel the pending point.

public void CancelPoint()

```
{  
    // If there is a pending point coroutine running...  
    if (pointCoroutine != null)  
    {  
        // ...stop it.  
        StopCoroutine(pointCoroutine);  
        // Clear the reference.  
        pointCoroutine = null;  
    }  
}
```

```
}  
}  
}
```

6. Pointer.cs

```
// Imports the necessary Unity library.  
using UnityEngine;
```

```
// Defines the Pointer class.
```

```
public class Pointer : MonoBehaviour  
{
```

```
    // A private flag to ensure points are awarded only once per throw.  
    private bool hasScoredThisThrow = false;
```

```
    // Unity's OnTriggerEnter function, runs when something enters this  
    object's trigger collider.
```

```
    void OnTriggerEnter(Collider other)  
    {
```

```
        // If the object that entered has the "Player" tag AND hasn't  
        scored yet this throw...
```

```
        if (other.gameObject.CompareTag("Player") &&  
!hasScoredThisThrow)
```

```
        {
```

```
            // ...set the flag to true so it can't score again.
```

```
            hasScoredThisThrow = true;
```

```
            // Tell the collisions script to cancel any pending board point.  
            collisions.instance.CancelPoint();
```

```
            // Tell the GameManager to add 3 points.
```

```
            GameManager.instance.AddScore(3);
```

```
        }
```

```
    }
```

```
}
```

7. groundhit.cs

```
// Imports the necessary Unity library.  
using UnityEngine;
```

```
// Defines the groundhit class.
```

```
public class groundhit : MonoBehaviour  
{
```

```
    // Unity's OnCollisionEnter function, runs when something hits the  
    ground.
```

```
    void OnCollisionEnter(Collision collision)
```

```
{
```

```
    // If the object that hit has the "Player" tag...
```

```
    if (collision.gameObject.CompareTag("Player"))
```

```
{
```

```
    // ...tell the collisions script to cancel any pending board point.
```

```
    collisions.instance.CancelPoint();
```

```
}
```

```
}
```

```
}
```

Appendix C: Project Dictionary

This section defines the key programming and Unity concepts you've learned by building this game.

- **AudioClip:** An audio file (like an MP3 or WAV) that can be played in the game. It's the "song."
- **AudioSource:** The component that acts as a speaker. You give it an AudioClip to play.
- **Awake():** A built-in Unity function that runs once per object, as

soon as it's loaded into the scene. It runs before `Start()`. Ideal for setting up references.

- **Boolean (bool):** A variable type that can only be true or false. Used as a switch to track states like `gameOver`.
- **Canvas:** The UI element that holds all other UI components like text and buttons. It's the drawing board for your interface.
- **Class:** A blueprint for creating objects. In Unity, every script file is a class that defines a new type of component.
- **Collider:** An invisible shape that Unity uses to detect physical collisions between objects.
- **Conditional Logic (if/else):** The primary way to make decisions in code. if a condition is true, do something; else, do something different.
- **Coroutine:** A special function that can pause its execution and resume later. Perfect for animations, delays, or tasks that unfold over several frames. Started with `StartCoroutine()`.
- **Destroy():** A command to remove a `GameObject` or component from the game.
- **Float:** A variable type for numbers that can have decimal points (e.g., `1.5f`, `-10.25f`). Used for things like time and speed.
- **Function (or Method):** A named block of code that performs a specific task. You can "call" a function to run its code from other places.
- **GameObject:** The fundamental object in Unity. Everything in your scene, from characters to lights to managers, is a `GameObject`. Think of them as empty containers.
- **GetComponent<>():** A command to find and get a reference to another component on the same `GameObject`.
- **Input:** The Unity class used to read input from the player, such as mouse clicks (`Input.GetMouseButtonDown`) or screen touches.
- **Inspector:** The window in the Unity Editor where you can view and edit the properties (variables) and components of a selected

GameObject.

- **Instantiate():** A command to create a new copy of a *GameObject* (usually a *Prefab*) during the game.
- **Integer (int):** A variable type for whole numbers (e.g., 1, 5, -10). Used for scores and counts.
- **Material:** An asset that defines how the surface of a 3D model is rendered. We use it to change the color of the bags.
- **MonoBehaviour:** The base class that every Unity script must inherit from. It's what allows your script to be attached to a *GameObject* as a component.
- **Namespace (using):** A way to organize code. The using directive at the top of a script tells *C#* which libraries of code (like *UnityEngine* or *TMPro*) you want to use.
- **Prefab:** A "master copy" of a *GameObject* that is saved in your project files. You can use it to create many identical instances of that object. Our *Cornrag* is a *prefab*.
- **Private:** A keyword that makes a variable or function accessible only from within the same script (class). It won't show up in the *Inspector*.
- **Public:** A keyword that makes a variable or function accessible from other scripts and makes variables visible in the *Inspector* so you can edit them.
- **Random.Range():** A function that returns a random number between a minimum and maximum value. Used for the bot's inaccuracy.
- **Rigidbody:** A component that tells Unity to apply physics to a *GameObject*. It allows the object to be affected by forces and gravity.
- **Singleton Pattern:** A design pattern used for manager scripts. It ensures there is only ever one instance of that manager in the scene and provides an easy, global way to access it (e.g., *GameManager.instance*).
- **Start():** A built-in Unity function that runs once per script, just

before the first frame of the game is updated. It runs after `Awake()`.

- **String:** A variable type for text (e.g., "Player 1 Wins!").
- **Tag:** A label you can assign to one or more `GameObjects`. It's an easy way to check what kind of object you've collided with (e.g., `CompareTag("Player")`).
- **TextMeshPro (TMP):** The advanced text rendering system in Unity used for all our UI.
- **Transform:** A fundamental component on every `GameObject` that stores its Position, Rotation, and Scale in 3D space.
- **Trigger:** A special type of collider that doesn't cause a physical collision but instead detects when another object enters its volume. Used for the hole.
- **Update():** A built-in Unity function that is called once every single frame. It's where most real-time game logic, like checking for input, happens.
- **Vector2 / Vector3:** A data structure (struct) that holds two (x, y) or three (x, y, z) numbers. Used to represent positions, directions, and forces in 2D or 3D space.